# I2C 센서 모니터링 시스템

## Basys3 Master·Slave

DHT11 · HC-SR04 · SSD1306

발표자 : 이규현 · 박수민

# CONTENTS


01 서론 및 개요

02 Block Diagram

03 FSM

04 Source Code

05 Test Bench · 실측 파형

06 동작 동영상

07 Troubleshooting

08 결론 및 구현 후기

01

# 시스템 개요

센서 측정은 Slave, I2C 요청과 출력 제어는 Master가 담당합니다

## Slave Basys3

DHT11·HC-SR04 자동 측정
스냅샷 레지스터와 I2C 응답
로컬 FND·UART

## Master Basys3

100ms마다 0x42 자동 읽기
거리 16비트 복원과 BCD 변환
FND·UART·OLED 출력

## SSD1306 OLED

주소 0x3C / 128×32
HUM·TEMP·DIST + 애니메이션
동일한 SDA·SCL 버스 사용

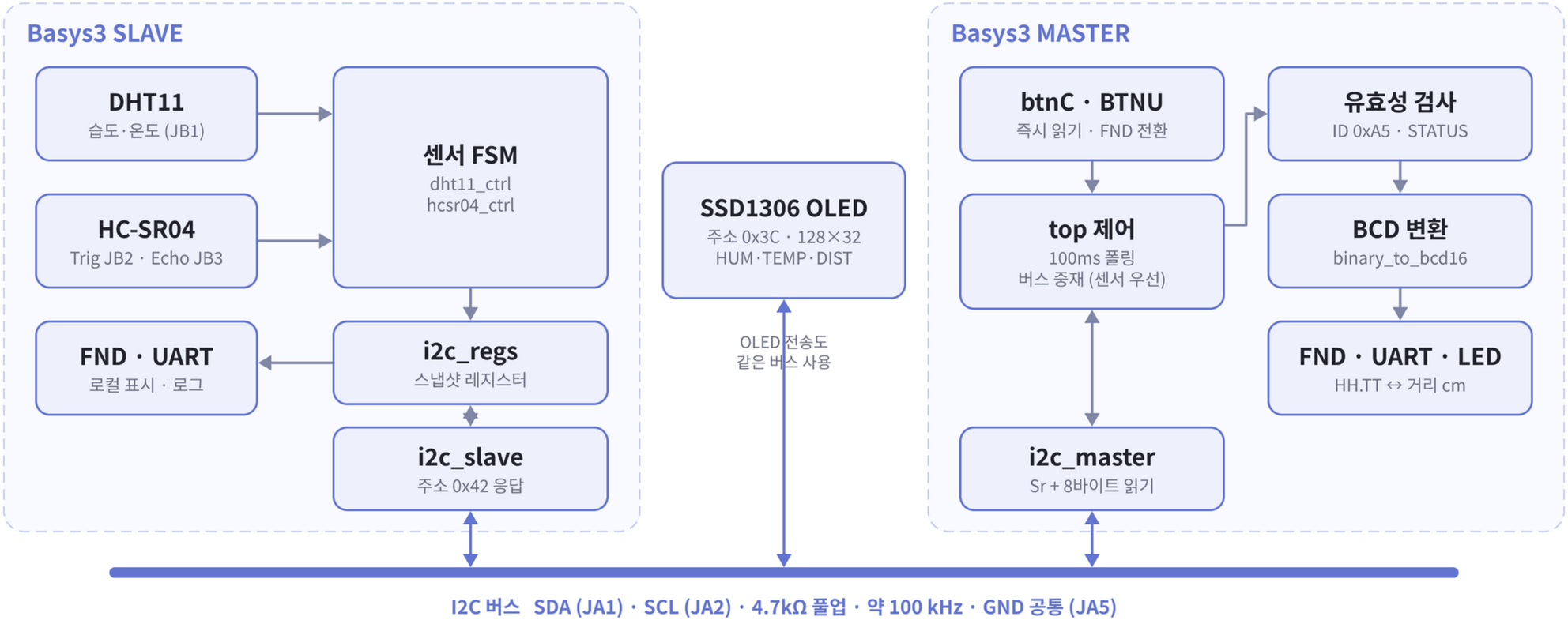
**100 MHz**
시스템 클럭

**약 100 kHz**
I2C 통신

**100ms 자동 읽기**
btnC 즉시 읽기 · BTNU FND 전환

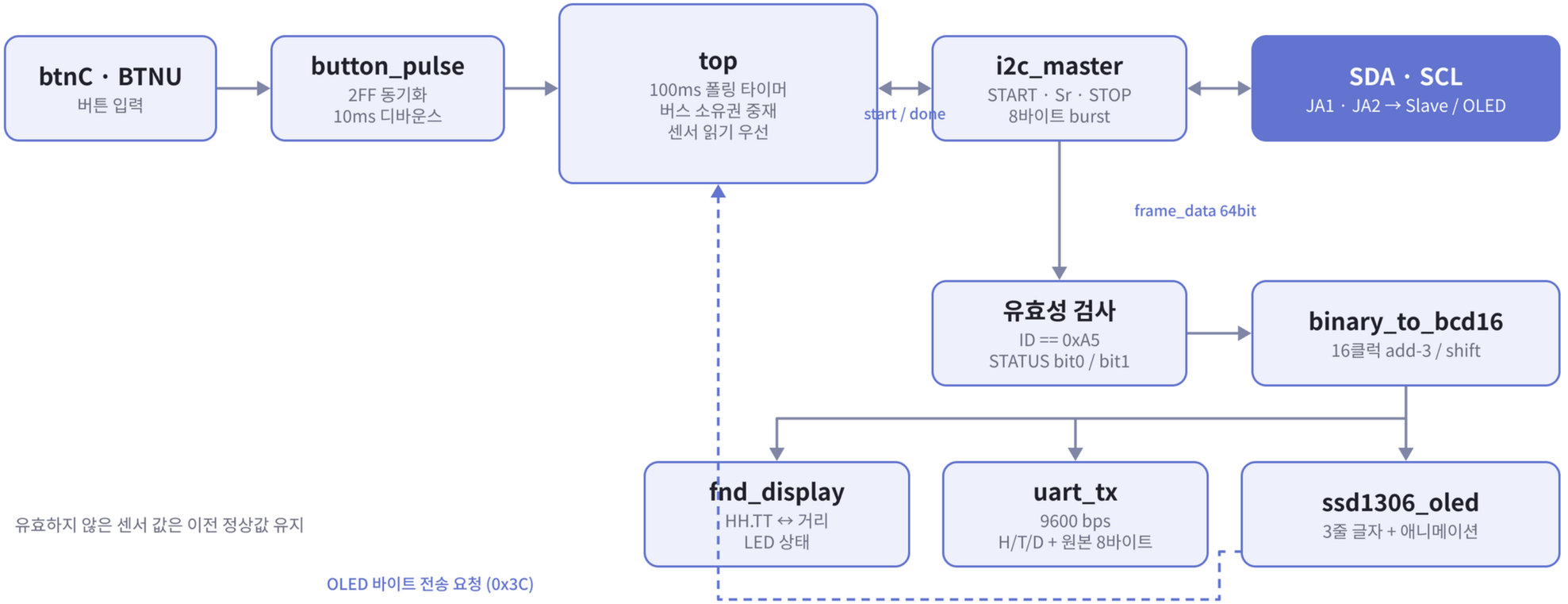
# 02
# 전체 시스템 구성

두 I2C 장치가 SDA·SCL을 공유하고 주소로 접근 대상을 구분합니다



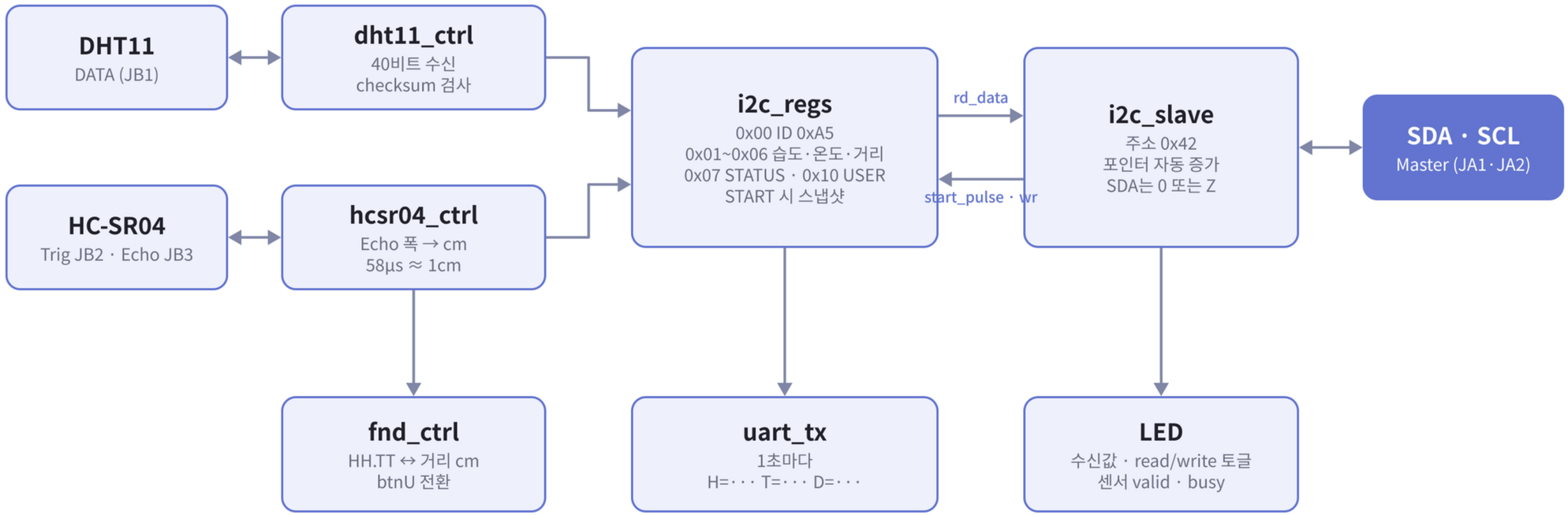
# 02
# Master 내부 구성

하나의 I2C 코어를 센서 요청과 OLED 전송 공유



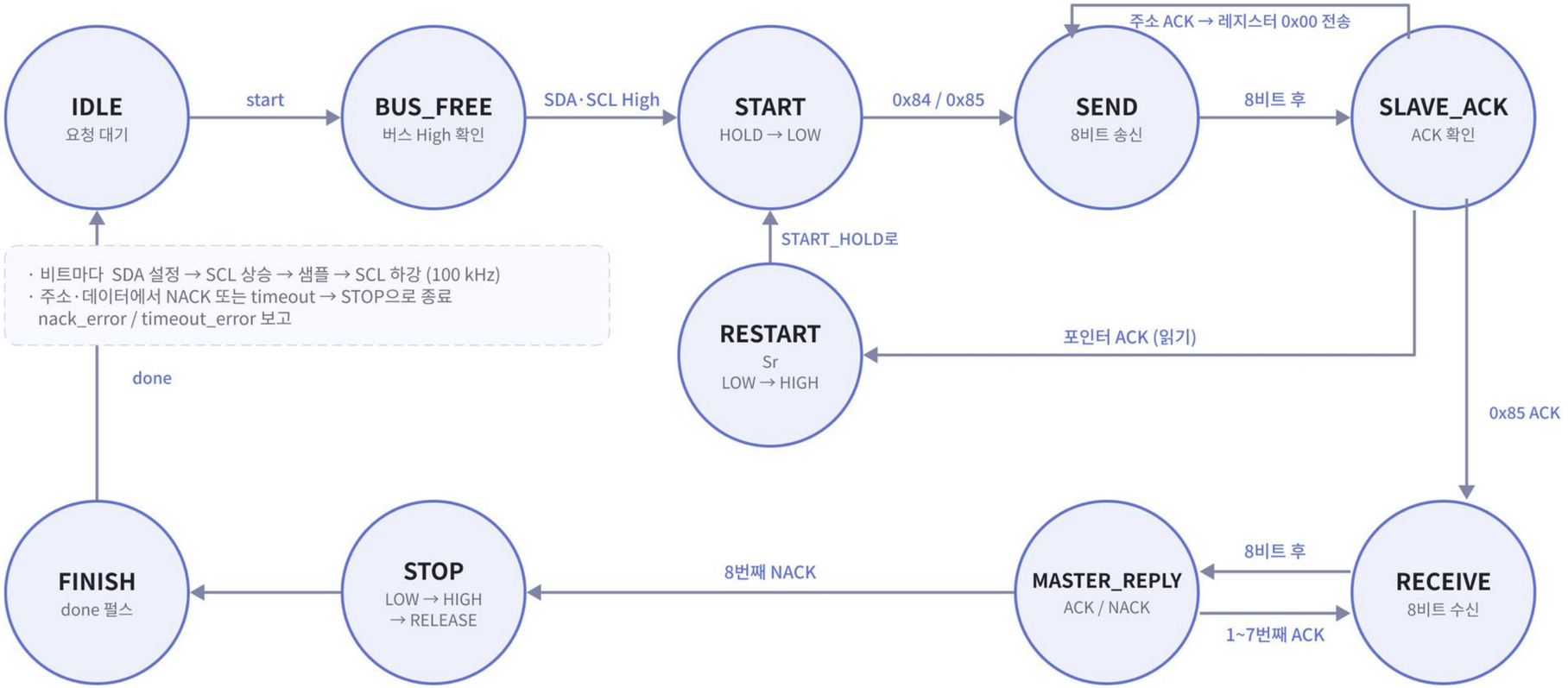
# 02
# Slave 내부 구성

live 센서값은 로컬 출력에, 스냅샷 값은 I2C 응답에 사용합니다



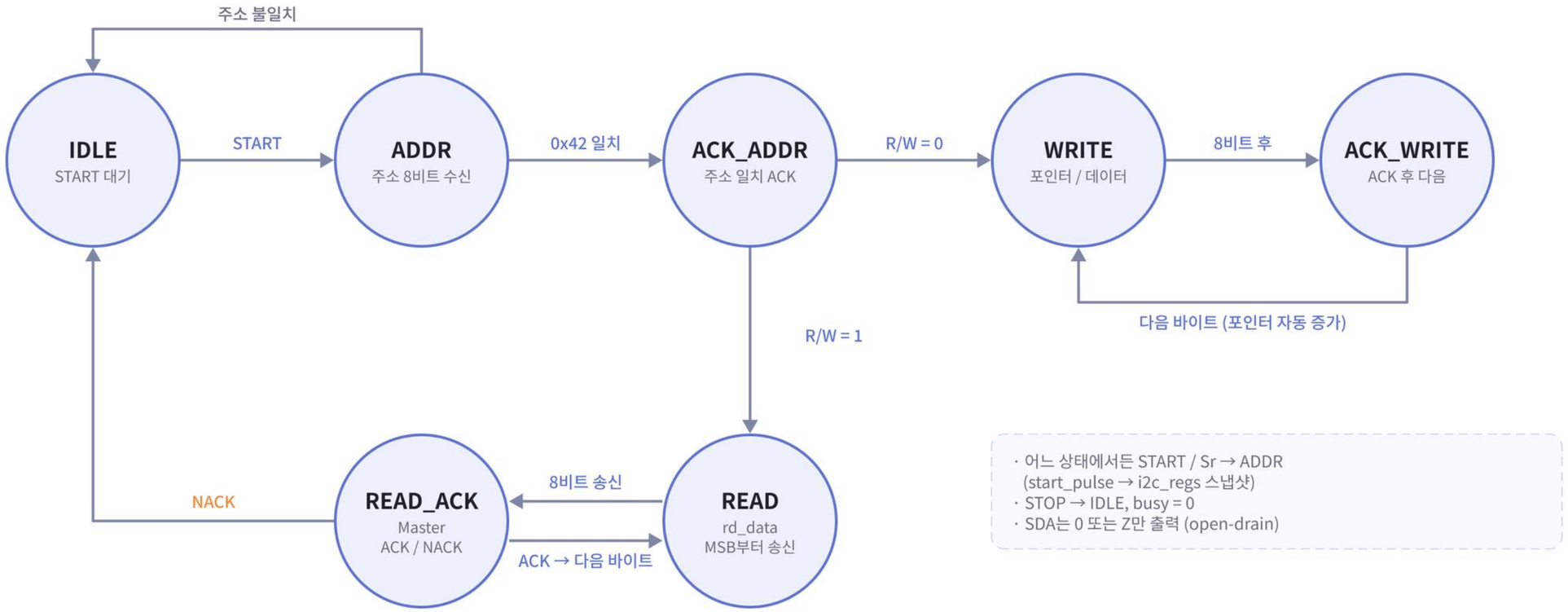
# 03
# I2C Master FSM

0x00 포인터 지정 후 repeated START로 8바이트 읽음



# 03
# I2C Slave FSM

주소 0x42 판별 후 ACK하고, R/W 비트에 따라 읽기와 쓰기를 처리




- 어느 상태에서든 START / Sr → ADDR (start_pulse → i2c_regs 스냅샷)
- STOP → IDLE, busy = 0
- SDA는 0 또는 Z만 출력 (open-drain)

# 03
# Slave 센서 FSM

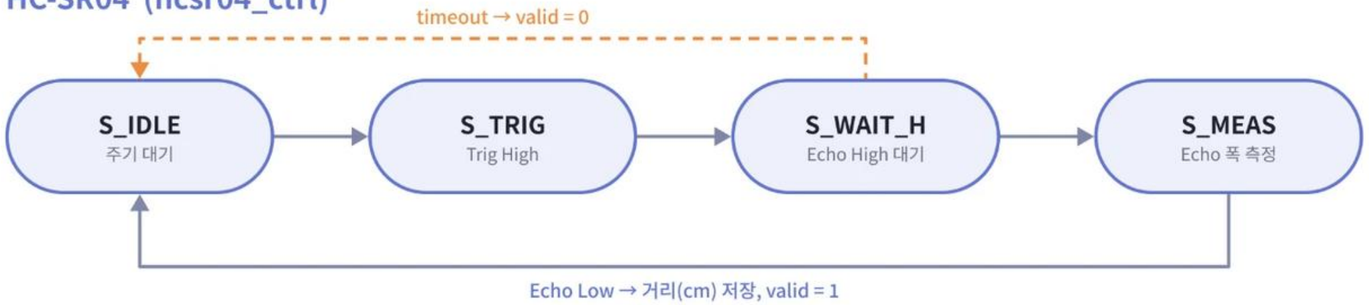
DHT11과 HC-SR04는 별도 FSM으로 주기 측정

## DHT11 (dht11_ctrl)




- 단계마다 200µs timeout → S_DONE
- checksum 일치 → valid = 1
  습도·온도 정수/소수 갱신
- 실패 시 valid = 0, 이전 값 유지

## HC-SR04 (hcsr04_ctrl)

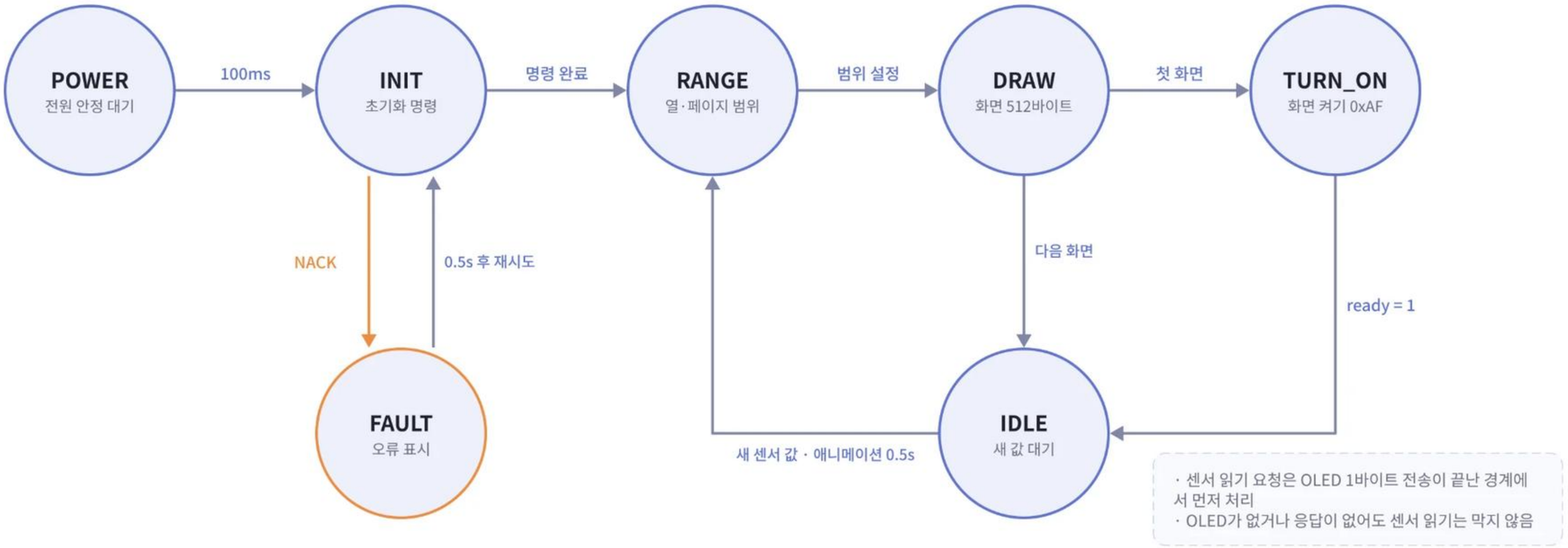



- Echo High 58µs ≈ 1cm
- 16비트 거리 → 0x05·0x06
- 30ms 안에 응답 없으면 무효

03

# OLED FSM · 공유 버스 중재

센서 읽기 요청은 OLED 바이트 전송 경계에서 우선 처리



04

# Slave 스냅샷 레지스터

START마다 센서값을 보관해 하나의 burst 동안 동일한 값을 사용

i2c_regs.v

```
// START / Sr 순간 센서값 고정 → burst 중 값이 섞이지 않음
if (snapshot) begin
    s_hum_int <= hum_int;
    s_hum_dec <= hum_dec;
    s_tmp_int <= tmp_int;
    s_tmp_dec <= tmp_dec;
    s_dist    <= dist_cm;
    s_status  <= {dist_valid, dht_valid};
end
```

## 스냅샷 목적

8바이트를 읽는 중
센서값이 바뀌어도
수신 값이 섞이지
않도록 보관

**start_pulse → snapshot**

04

# 레지스터 맵과 8바이트 읽기

Slave 주소 0x42 · 100ms마다 포인터 0x00부터 8바이트를 순서대로 읽음

| 주소 | 데이터 | Master에서 사용 |
|---|---|---|
| 0x00 | ID = 0xA5 | 장치 식별 |
| 0x01–02 | 습도 정수 / 소수 | HUM 표시 |
| 0x03–04 | 온도 정수 / 소수 | TEMP 표시 |
| 0x05–06 | 거리 MSB / LSB | 16비트 거리(cm) |
| 0x07 | STATUS bit0 / bit1 | 센서별 유효 판정 |
| 0x10 | USER | 사용 안 함 (Slave LED 표시용) |

top.v

```
// 0x05(상위) · 0x06(하위) 두 바이트를 16비트 거리로 복원
wire [15:0] completed_distance = {frame_data[47:40], frame_data[55:48]};
```

04

# 센서 유효성 검사 · 버스 중재

DHT11과 초음파 거리의 유효 플래그를 따로 처리

top.v

```
wire id_valid       = frame_data[7:0] == 8'ha5;     // WHO_AM_I
wire frame_valid    = id_valid && frame_data[56];   // STATUS bit0 : DHT11
wire distance_valid = id_valid && frame_data[57];   // STATUS bit1 : 초음파
```

## STATUS bit0

DHT11 체크섬 유효
습도·온도만 갱신

## STATUS bit1

초음파 거리 유효
거리만 갱신 가능

## 오류 처리

ID 불일치 / NACK / timeout
기존 정상 표시값 유지

04

# 초음파 거리 수신·BCD 변환

0x05·0x06 결합 후 16클럭 add-3/shift로 변환하고 출력 장치가 공유합니다

**01 Slave 레지스터**

0x05 상위 / 0x06 하위
STATUS bit1 거리 유효

→

**02 Master 읽기**

포인터 0x00 / Sr
8바이트 수신

→

**03 유효성 검사**

ID=0xA5 + STATUS bit1
무효값은 이전 값 유지

↓

**04 16비트 복원**

{상위 바이트, 하위 바이트}
0x01·0x2C = 300cm

→

**05 BCD 변환**

binary_to_bcd16
16클럭 / 5자리 BCD

→

**06 출력**

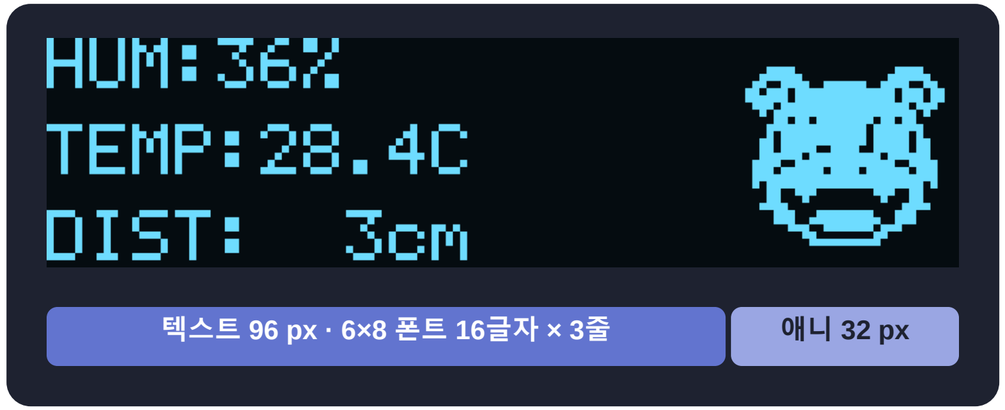
OLED 999cm · FND 9999 제한
UART: 원본 거리 5자리

top.v

```
binary_to_bcd16 u_distance_bcd (
    .clk(clk), .reset(rst), .start(completed_read), .binary(completed_distance),
    .bcd(converted_distance), .done(distance_converted)
);
```

# 04
# OLED 문자 출력

습도·온도(소수 1자리)·거리 3줄 + 오른쪽 32×32 애니메이션 · 128×32




ssd1306_oled.v

```
// OLED 거리는 999cm 제한, 수신값은 그대로 유지
wire [11:0] shown_distance =
    (draw_distance[19:12] != 0)
        ? 12'h999 : draw_distance[11:0];
```

거리는 화면에서 최대 999 cm로 제한 → 4자리 값이 와도 줄이 넘치지 않음

### 화면 구성

128×32 = 4 page × 128 col = 512 byte
텍스트 3줄을 y = 0 · 12 · 24에 배치
오른쪽 32×32는 그림 영역

### 값 표시 형식

HUM 정수 %
TEMP 정수.소수 1자리 C
DIST 0~999 cm · 센서 오류 시 "--"

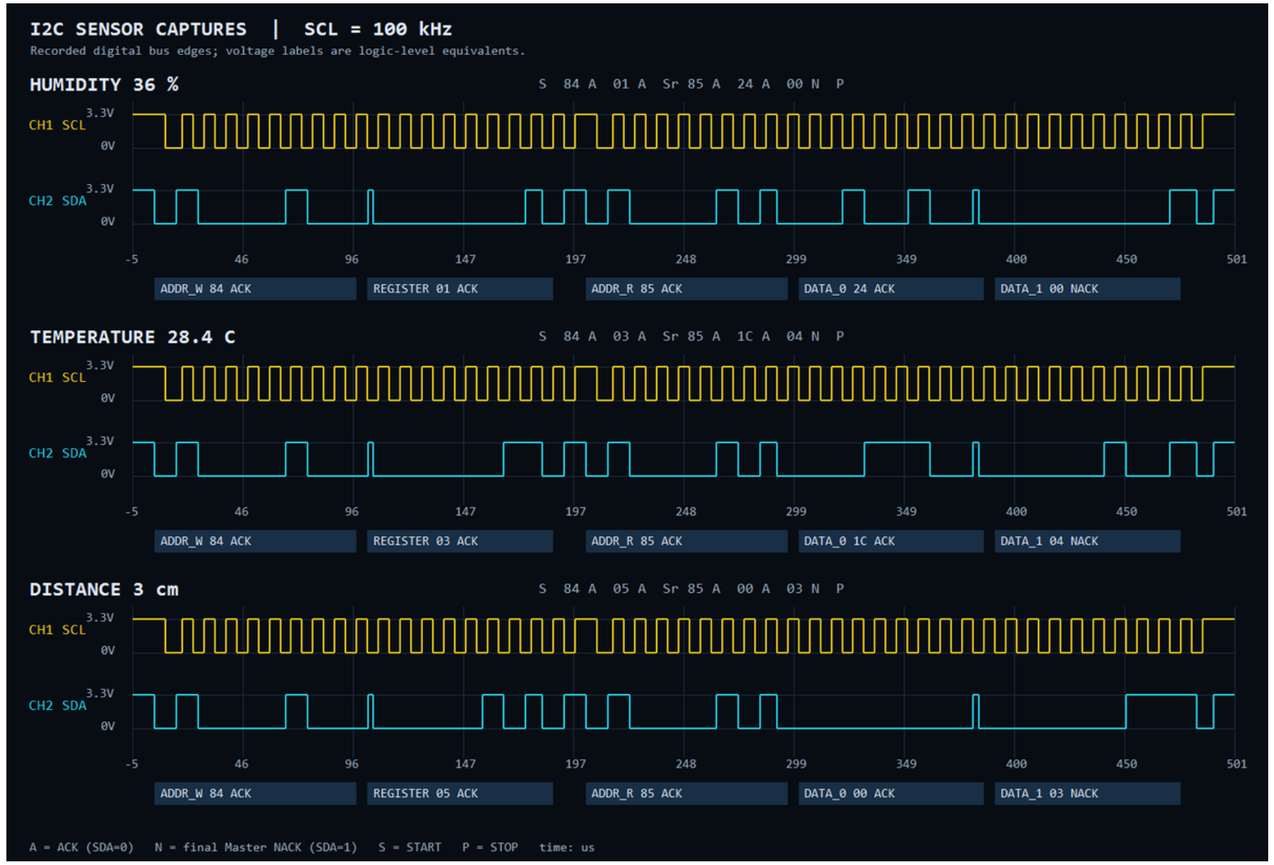
### 애니메이션

4프레임을 0.5 s마다 교체
프레임만 바뀌면 오른쪽 32열만 다시 전송
→ 센서 읽기와 버스 충돌 최소화

05

# Master 테스트벤치 · 센서별 읽기

tb_i2c_scope : 실제 Slave RTL과 연결해 레지스터를 센서별로 2바이트씩 읽은 시뮬레이션 (SCL 100 kHz)




## 테스트벤치 구성

tb_i2c_master
쓰기·읽기 전송, reset 복구

tb_top
실제 Slave 읽기, BTNU 전환, FND·UART, 오류 복구

tb_top_polling
100ms 주기 읽기, OLED/UART 중 버스 중재

tb_oled
온도 소수 표시, 글자 픽셀, 애니메이션

tb_i2c_scope
습도 36% · 온도 28.4℃ · 거리 3cm 파형

05

# Slave 응답·스냅샷 검증

I2C Master 모델로 Slave를 단독 검증 · tb_i2c_slave 18개 check 모두 PASS

| 검사 항목 | tb_i2c_slave 검증 내용 |
| --- | --- |
| 주소 / 포인터 | 0x42 ACK, 0x00 설정 및 repeated START |
| 연속 읽기 | 8바이트 순서, ACK / 최종 NACK |
| burst 읽기 | 0x01부터 7바이트 값 일치 (45 / 0 / 23 / 1 / 0x0123 / 0x03) |
| USER 쓰기 | 0x10에 0x5A 쓰기, last_rx·LED 토글 확인 |
| 오류 / 복구 | 잘못된 주소 0x43 NACK 후 재읽기, STOP 후 busy 해제 |

05

# 합성 / 구현 결과

Vivado 2020.2 · xc7a35tcpg236-1 · 100 MHz 제약 · 두 보드 모두 타이밍 통과

**Master** LUT 997 · +0.069 ns　　**Slave** LUT 578 · +3.126 ns

## Master (top.bit)

| | |
|---|---|
| **Slice LUTs** | 997 (4.8%) |
| **Slice Registers** | 615 (1.5%) |
| **WNS (setup)** | +0.069 ns |
| **WHS (hold)** | +0.122 ns |
| **DRC** | 위반 0개 |

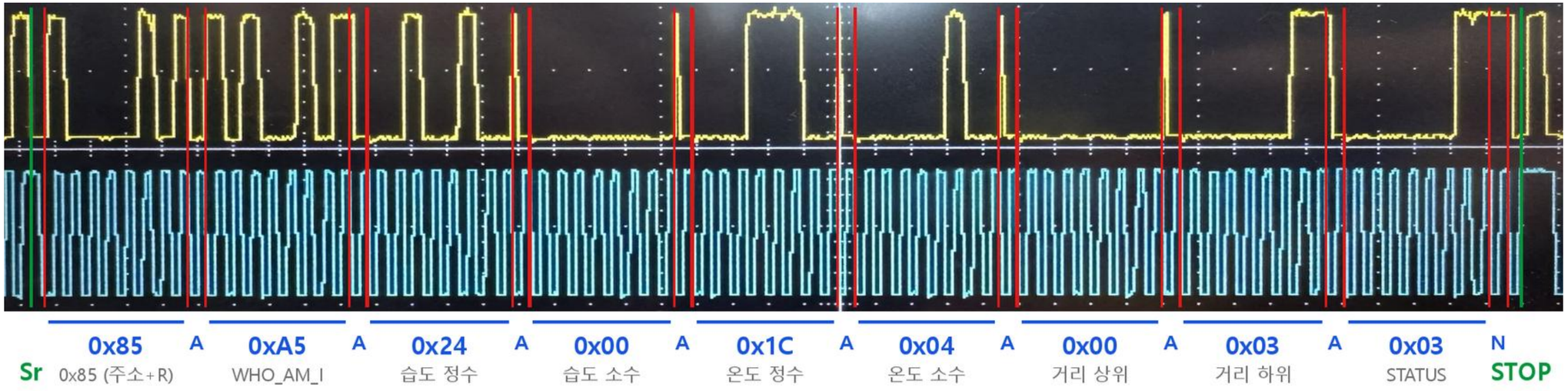
## Slave (top_slave.bit)

| | |
|---|---|
| **Slice LUTs** | 578 (2.8%) |
| **Slice Registers** | 475 (1.1%) |
| **WNS (setup)** | +3.126 ns |
| **WHS (hold)** | +0.090 ns |
| **Clock** | 100 MHz |

05

# 센서 읽기 1회 실측 파형

Master btnC 1회 → repeated START부터 STOP까지 9바이트를 한 화면에 캡처 (SCL 99.86 kHz)

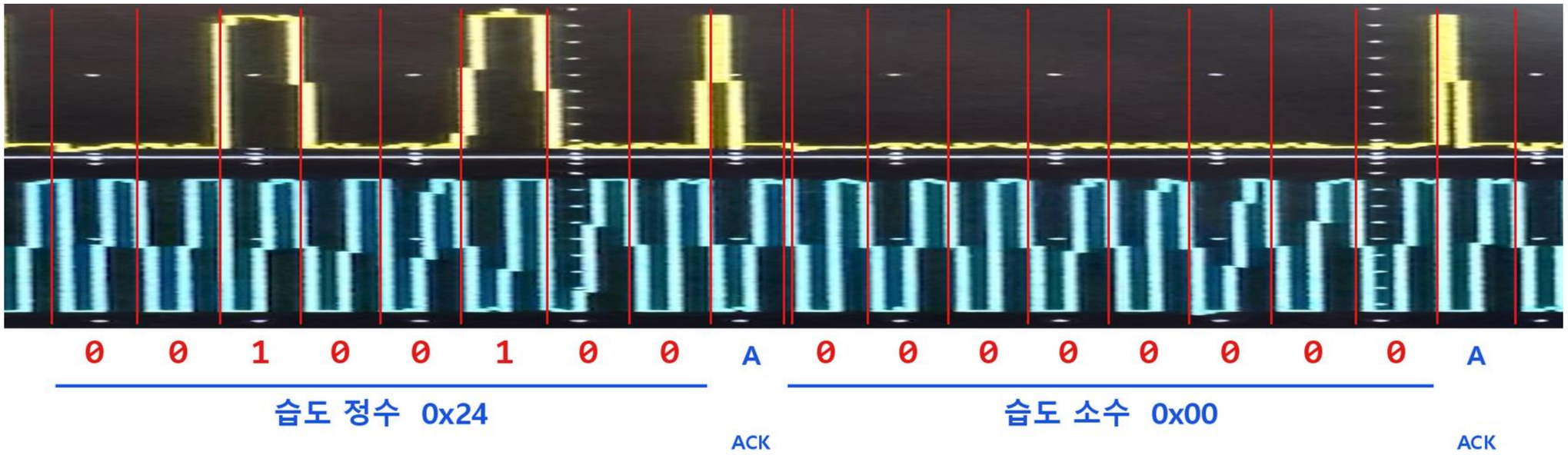


05

# 습도 파형 · Reg 0x01, 0x02

**36.0 %**

습도 정수 0010 0100 = 0x24 = 36 · 습도 소수 0000 0000 = 0x00 = 0

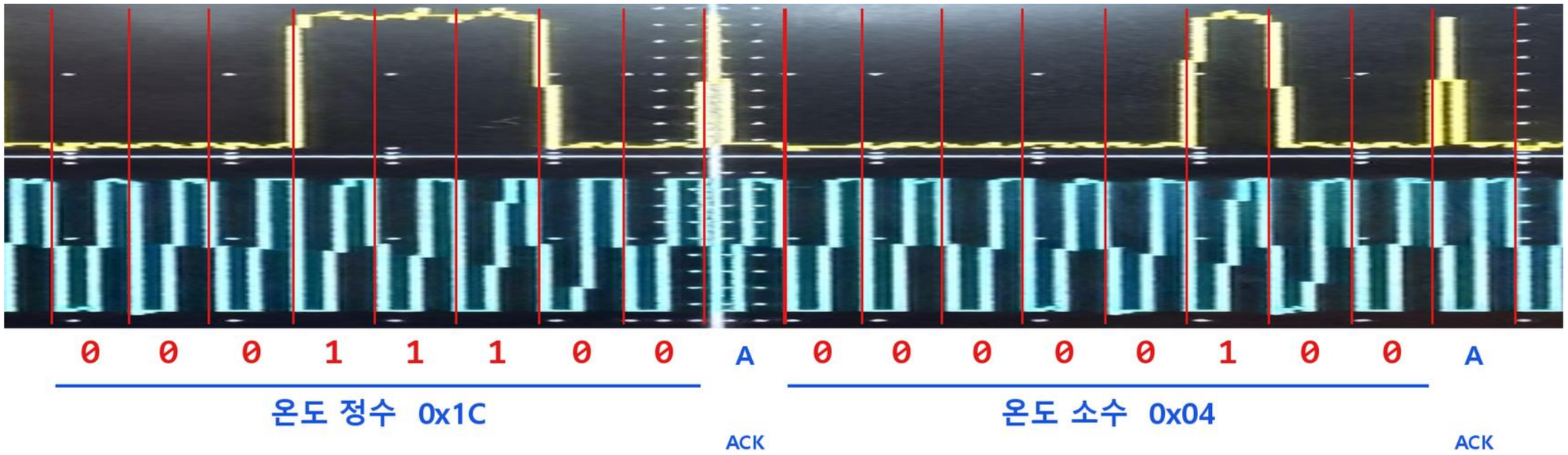


05

# 온도 파형 · Reg 0x03, 0x04

28.4 ℃

온도 정수 0001 1100 = 0x1C = 28 · 온도 소수 0000 0100 = 0x04 = 4

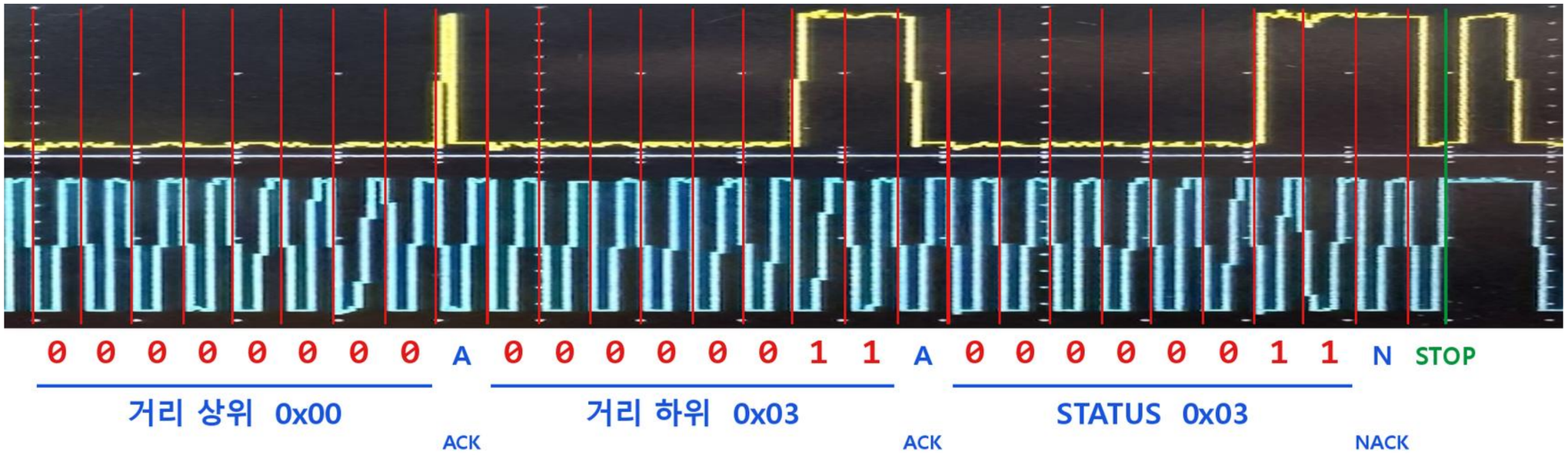


05

# 거리 파형 · Reg 0x05, 0x06

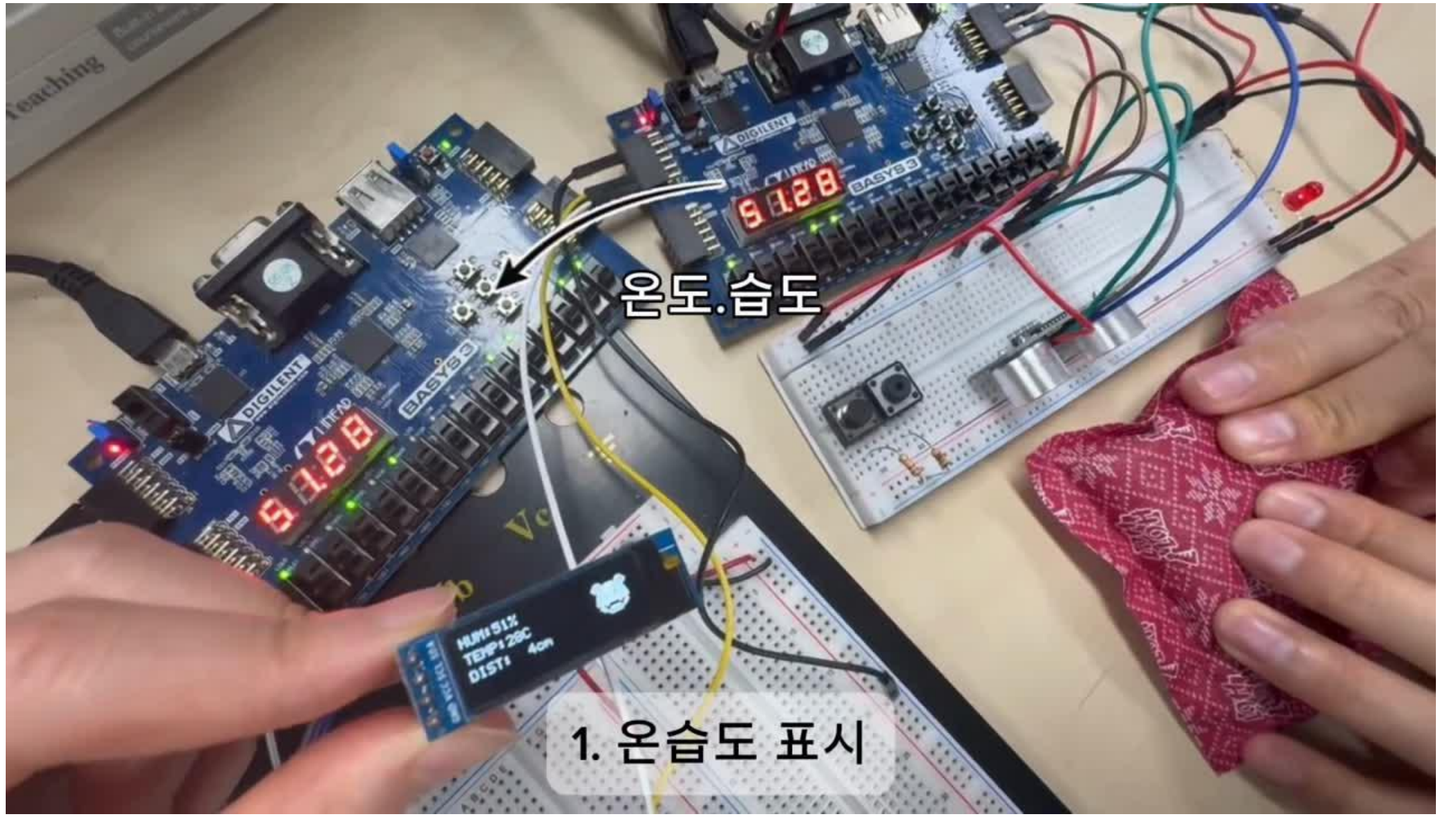
**3 cm**

거리 0000 0000 0000 0011 = 0x0003 = 3 cm · STATUS 0x03 = DHT11 OK · 초음파 OK



06

# 실물 동작 시연



# 07
# 레지스터 포인터·데이터 해석 문제

| 관찰 | 원인 확인 | 수정 |
| --- | --- | --- |
| 45 / 0 / 27 순서 표시 | 1바이트를 값 하나로 취급 | 8바이트 프레임으로 읽기 |
| A5 또는 0 표시 | ID·소수부 / 포인터 위치 혼동 | 매번 0x00 포인터 설정 |
| 반복 읽기값 이동 | Slave 포인터 자동 증가 | repeated START 주소 지정 읽기 |

### 8바이트 프레임

ID / 습도정수 / 습도소수 / 온도정수 / 온도소수 / 거리H / 거리L / STATUS

07

# 거리 BCD 변환과 타이밍

## 초기 구현

16비트 거리를 조합 나눗셈으로 십진 변환

**−6.687 ns**

Post-route WNS

## 수정 후

16클럭 add-3 / shift (double dabble)

**+0.069 ns**

최종 빌드 WNS (OLED·폴링 포함, WHS +0.122 ns)

# 08
# 정리 및 향후 개선

Master·Slave의 역할과 검증 범위 구분

01 **Slave**

DHT11·HC-SR04 측정, 스냅샷, I2C 응답, 로컬 출력

02 **Master**

100ms 자동 읽기·btnC, 유효성 검사, BCD, OLED·UART·FND

03 **검증**

Slave I2C·레지스터 연동 TB / 오실로스코프 실측 (SCL 99.86 kHz, 8바이트 읽기 확인)